

Day 17 — cross-validation and hyperparameter tuning

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 3, Day 3 — complete line-by-line companion to `day17_cross_validation.py`

Same format as the Day 18 lesson: every section explains the idea first, then shows the exact code, then walks through what each line does and why it's written that way. Read the explanation before the code — the code will make more sense once you already know what problem it's solving.

0. Imports and setup

Day 16 ended by comparing three models on one single train/test split, and the script itself warned that close differences could "shift with a different `random_state`". Day 17 exists to fix exactly that weakness: instead of trusting one lucky or unlucky split, evaluate every model several times and report a mean and a spread, then use that same discipline to search for good hyperparameters instead of guessing them.

```
import os
import urllib.request

import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

→ `cross_val_score` — new this script. Runs a model through K-fold cross-validation in one call and hands back an array of scores, one per fold

→ `GridSearchCV` — new this script. Automates trying every combination in a grid of hyperparameters, using cross-validation to score each combination fairly

→ the three model classes (`LogisticRegression`, `DecisionTreeClassifier`, `RandomForestClassifier`) are the exact same three from Day 16, imported together here so they can be cross-validated side by side in a loop

1. `prepare_data()` — same pipeline as Day 15-16

Identical cleaning and encoding logic carried forward unchanged: fill missing Age with the median, drop Cabin, fill missing Embarked with the mode, engineer FamilySize, one-hot encode Sex and Embarked. This script adds one important new idea in its print statements: it names `X_test` explicitly as data that stays "untouched until step 5".

```

def prepare_data(path=DATA_PATH, url=DATA_URL):
    if not os.path.exists(path):
        urllib.request.urlretrieve(url, path)

    df = pd.read_csv(path)
    df["Age"] = df["Age"].fillna(df["Age"].median())
    df = df.drop(columns=["Cabin"])
    df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
    df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

    y = df["Survived"]
    feature_cols = ["Pclass", "Sex", "Age", "SibSp", "Parch", "Fare",
                    "Embarked", "FamilySize"]
    X = pd.get_dummies(df[feature_cols], columns=["Sex", "Embarked"], drop_first=True)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    return X_train, X_test, y_train, y_test

```

→ `df["Age"].fillna(df["Age"].median())` — median, not mean, because a few very old passengers would otherwise pull the average upward

→ `pd.get_dummies(..., drop_first=True)` — converts Sex into a single Sex_male column and Embarked into two columns; `drop_first=True` drops one redundant category per feature since it can always be inferred from the others

→ `test_size=0.2, random_state=42, stratify=y` — 20% held out for testing, a fixed seed so this exact split is reproducible, and `{C}stratify=y{E}` keeps the same survival rate in both pieces

→ this `X_train/X_test/y_train/y_test` split is deliberately created once, here, and then reused everywhere else in the script — every cross-validation and grid search below runs only on `X_train/y_train`, never touching `X_test/y_test` until the very last function

2. demonstrate_split_instability() — proving one split isn't enough

Before introducing cross-validation as the fix, the script proves there's a problem to fix. It trains the exact same logistic regression on 5 different random splits of the training data and shows the accuracy changing each time — same model, same code, same original data, different result purely from how the rows happened to land.

```
def demonstrate_split_instability(X_train, y_train):
    accs = []
    for seed in range(5):
        X_a, X_b, y_a, y_b = train_test_split(
            X_train, y_train, test_size=0.25, random_state=seed, stratify=y_train
        )
        model = LogisticRegression(max_iter=1000)
        model.fit(X_a, y_a)
        acc = accuracy_score(y_b, model.predict(X_b))
        accs.append(acc)

    print(f"Range across 5 different splits: {min(accs):.3f} to {max(accs):.3f}")
```

→ `for seed in range(5)` — loops through 5 iterations, using seed (0, 1, 2, 3, 4) as a different `random_state` each time, so each pass produces a genuinely different split

→ `train_test_split(X_train, y_train, ...)` — note this splits `X_train` again, not the original `X`. This whole demonstration happens entirely inside the training data; `X_test` is never involved, because this is just illustrating a concept, not evaluating a final model

→ `X_a, X_b, y_a, y_b` — deliberately named differently from `X_train/X_test` to make clear these are temporary sub-splits used only inside this one function, not the real train/test split from `prepare_data()`

→ `accs.append(acc)` — builds up a plain Python list of 5 accuracy numbers across the loop; `min(accs)` and `max(accs)` afterward show the full spread in one line

3. `run_cross_validation()` — K-fold CV on all three models

What K-fold cross-validation actually does

Split the training data into K equal-sized folds. Train on K-1 of them, test on the one left out, and repeat K times so every fold gets a turn being the test set. With 5-fold CV, every row is used for testing exactly once and for training four times — the final score is not one number but K numbers, one per fold, which is far more informative than a single split ever could be.

```
def run_cross_validation(X_train, y_train):
    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000),
        "Decision Tree (max_depth=4)": DecisionTreeClassifier(
            max_depth=4, min_samples_leaf=10, random_state=42
        ),
        "Random Forest (default)": RandomForestClassifier(n_estimators=200, random_state=42)
    }

    results = {}
    for name, model in models.items():
        scores = cross_val_score(model, X_train, y_train, cv=5)
        results[name] = scores
        print(f" mean = {scores.mean():.3f} +/- std = {scores.std():.3f}")

    return results
```

→ `models = {...}` — a dictionary mapping a human-readable name (used later for printing) to an actual unfitted model instance. Notice none of these three models has been `.fit()` yet — `cross_val_score` handles fitting internally, once per fold

→ `for name, model in models.items()` — `.items()` unpacks a dictionary into key/value pairs so the loop can use both the readable name and the actual model object on each pass

→ `cross_val_score(model, X_train, y_train, cv=5)` — internally splits `X_train` into 5 folds, trains a fresh copy of `{C}model{E}` 5 separate times (once per fold combination), and returns an array of 5 test scores — no manual splitting loop needed

→ `results[name] = scores` — stores the full 5-value array (not just its mean) under this model's name in the results dictionary, so later code can look up e.g. `results["Random Forest (default)"]` by that exact string

→ critical indentation detail: `return results` sits at the same indentation level as the `for` loop above it, not inside the loop body. If it were indented one level deeper (inside the loop), the function would exit after processing only the first model and the other two would never run

4. `run_grid_search()` — automating the hyperparameter search

Hyperparameters vs learned parameters

A model's parameters (like logistic regression's coefficients) are learned automatically during `.fit()`. Hyperparameters — `max_depth`, `n_estimators`, `min_samples_leaf` — are settings chosen before training even starts, and picking good ones by hand means guessing. `GridSearchCV` turns that guess into an exhaustive, cross-validated search.

```
def run_grid_search(X_train, y_train):
    param_grid = {
        "n_estimators": [100, 200],
        "max_depth": [4, 8, None],
        "min_samples_leaf": [1, 5],
    }

    forest = RandomForestClassifier(random_state=42)
    search = GridSearchCV(forest, param_grid, cv=5, n_jobs=-1)
    search.fit(X_train, y_train)

    print(f"Best params: {search.best_params_}")
    print(f"Best CV score: {search.best_score_:.3f}")

    return search.best_estimator_, search.best_score_
```

→ `param_grid` — a dictionary where each key is a hyperparameter name (must match the model's actual constructor argument names exactly, e.g. `min_samples_leaf`, not `min_sample_leaf`) and each value is a list of settings to try

→ 2 values for `n_estimators` × 3 for `max_depth` × 2 for `min_samples_leaf` = 12 total combinations; `GridSearchCV` will run 5-fold CV on every single one of those 12, meaning 60 total model fits happen inside this one function call

→ `RandomForestClassifier(random_state=42)` here is deliberately created with no other hyperparameters set — `GridSearchCV` overrides them combination by combination during the search

→ `GridSearchCV(forest, param_grid, cv=5, n_jobs=-1)` — wraps the base model and the grid together; `n_jobs=-1` tells it to use every available CPU core in parallel instead of testing combinations one at a time

→ `search.fit(X_train, y_train)` — this single call runs the entire grid search; afterward `search.best_params_` holds the winning combination and `search.best_estimator_` holds an already-fitted model using those winning settings

5. `compare_tuned_vs_default()` — the honest comparison, then the one real test

This function does two separate jobs: first, compare the tuned model's cross-validation score against the untuned default random forest's cross-validation score, reported plainly even if the tuned version barely wins. Second — and only after every tuning decision has already been locked in — check the winning model against the held-out test set for the first and only time in the whole script.

```
def compare_tuned_vs_default(cv_results, tuned_cv_score, best_model,
                             X_train, y_train, X_test, y_test):
    default_forest_scores = cv_results["Random Forest (default)"]
    diff = tuned_cv_score - default_forest_scores.mean()

    if abs(diff) < 0.01:
        print("essentially no improvement from tuning on this dataset")
    else:
        print(f"tuning {'helped' if diff > 0 else 'hurt'} noticeably")

    best_model.fit(X_train, y_train)
    test_acc = accuracy_score(y_test, best_model.predict(X_test))
    print(f"Final check on the held-out test set: {test_acc:.3f}")
```

→ `cv_results["Random Forest (default)"]` — looks up the array of 5 scores stored earlier under that exact string key. This is why the dictionary key name in step 3 has to match character-for-character wherever it's read back later

→ `abs(diff) < 0.01` — treats any difference smaller than one percentage point as "no real improvement" rather than claiming a win from what's likely just noise

→ the ternary expression `'helped' if diff > 0 else 'hurt'` picks one of two words inline based on the sign of `diff`, avoiding a longer if/else just to choose which word to print

→ `best_model.fit(X_train, y_train)` — `GridSearchCV`'s `best_estimator_` was already fit during the search, but refitting here on the full `X_train` (not a CV fold) gives the final model every row of training data to learn from before its one real test

→ `accuracy_score(y_test, best_model.predict(X_test))` — this is the only line in the entire script that touches `X_test` or `y_test`. Every decision above it — which model looked best, which hyperparameters to use — was made using cross-validation on training data alone, so this final number is a genuinely honest, unpeeked-at estimate of real-world performance

6. Reading CV results honestly: mean and spread together

Always report both the mean and the standard deviation across folds (" 0.72 ± 0.04 "), never the mean by itself. A model scoring 0.81 ± 0.01 is more dependable than one scoring 0.81 ± 0.09 , even though their averages are identical — the second one is far more likely to disappoint on a new batch of data, and the standard deviation is the only number in this script that reveals that difference.

7. main() — the call order matters

```
def main():
    X_train, X_test, y_train, y_test = prepare_data()
    demonstrate_split_instability(X_train, y_train)
    cv_results = run_cross_validation(X_train, y_train)
    best_model, tuned_cv_score = run_grid_search(X_train, y_train)
    compare_tuned_vs_default(cv_results, tuned_cv_score, best_model,
                             X_train, y_train, X_test, y_test)
```

→ the order here is the entire point of the script: instability is demonstrated first (the problem), then cross-validation runs (the fix), then grid search tunes using that same fix, and only the very last call ever receives `X_test/y_test`

→ `cv_results` produced by step 3 is passed directly into step 5's `compare_tuned_vs_default` call, which is how that function is able to look up `cv_results["Random Forest (default)"]` without recomputing anything

8. Deliverable and push

```
git add day17_cross_validation.py
git commit -m "Day 17: Cross-validation and hyperparameter tuning - GridSearchCV, fold var
git push
```